

Reviewer Pack — Free Cumulant Sum Bounded by Disjointness Communication Comp...

Entry ecd8f062c47d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 01:18:26 UTC

Free Cumulant Sum Bounded by Disjointness Communication Complexity

Entry ID: ecd8f062c47d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 01:18:26 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a $|X| \times |Y|$ matrix M defining a partial function f , let $\mu(M) = \sum_{k \geq 1} |c_k(M)|$ where $c_k(M)$ are the free cumulants of M 's entries. Then $\mu(M) \leq O(\sqrt{R(f)})$ where $R(f)$ is the randomized communication complexity of f . For DISJ_n , $\mu(\text{DISJ}_n) = \Omega(n)$ holds.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures in matrix entries, which may correlate with the information-theoretic cost of coordinating disjointness checks. The sum's growth rate could reflect the minimal communication required to resolve entangled dependencies.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 83d43c94d7a5bc11

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def R_transform_inv(a):
        n = len(a)
        if n == 1:
            return a[0]
        b = [a[i] - sum(a[j] * b[j - i - 1] for j in range(i)) / (i + 1) for i in range(n)]
        c = [b[i] / (i + 1) for i in range(n)]
        return c

    def free_cumulants(M):
        n, m = len(M), len(M[0])
        R = [[0] * m for _ in range(n)]
        for i in range(n):
            for j in range(m):
                R[i][j] = M[i][j]
        for k in range(1, n + m - 1):
            for i in range(k + 1):
                j = k - i
                if i < n and j < m:
                    R[i][j] -= sum(R[x][y] * R[i - x][j - y] for x in range(i) for y in range(j))
        return [R_transform_inv(row) for row in R]

    def communication_complexity(n):
        # Simplified model of communication complexity for DISJ_n
        return n

    def generate_DISJ_n(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            M[i][i] = 1
        return M

    n = random.randint(5, 40)
    M = generate_DISJ_n(n)
```

```

cumulants = free_cumulants(M)
mu_M = sum(abs(c) for c in cumulants)
R_f = communication_complexity(n)

conjecture_holds = mu_M >= 0.9 * n
counterexample = "" if conjecture_holds else f"mu_M={mu_M} < 0.9n"

return {
    "metric_name": " $\mu(M)$ ",
    "metric_value": mu_M,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_mu_M = sum(res["metric_value"] for res in results) / len(results)
    std_mu_M = math.sqrt(sum((res["metric_value"] - mean_mu_M)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_mu_M} std={std_mu_M} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mu_M < 0.9n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38ecf086.py", line 75, in <module>
    result = run_trial(seed)
    ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38ecf086.py", line 54, in run_trial
    cumulants = free_cumulants(M)
    ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38ecf086.py", line 39, in free_cumulants
    R[i][j] -= sum(R[x][y] * R[i - x][j - y] for x in range(i) for y in range(j)) / (i * j)
    ~~~~~^

```

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to division by zero, preventing evaluation of conjecture | next: Fix implementation to handle zero division and retest with validated inputs

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	81549
2	propose	ollama_remote	qwen3:8b	0	0	112076
3	propose	ollama_remote	qwen3:8b	0	0	77388
4	propose	ollama_remote	qwen3:8b	0	0	113015
5	propose	ollama_remote	qwen3:8b	0	0	33506
6	preregistration	ollama_remote	qwen3:8b	0	0	24291
7	novelty	ollama_remote	qwen3:8b	0	0	21284
8	novelty	ollama_remote	qwen3:8b	0	0	13824
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10274
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8851
11	judge	ollama_remote	qwen3:8b	0	0	15664

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 511722 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/ecd8f062c47d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ecd8f062c47d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ecd8f062c47d.tar.gz (if generated)